
畳み込みニューラルネットワーク (CNN)

1 はじめに

手書き文字認識データセットである MNIST データセットを用いて，画像を説明変数として手書き文字認識を行う例を示した．

画像を説明変数として採用する方法にはいくつかあり，今回は畳み込みニューラルネットワーク (COnvolutional Neural Network, CNN) を用いる方法について実習する．

2 画像を説明変数とする方法

画像を説明変数とする方法には以下のような例がある。

- 画像のビット単位の情報を説明変数とする。
- CNNを用いて画像から抽出した特徴を説明変数とする。
- CNNに入力する前に，事前に画素の輝度差，コントラスト，形状情報等を数値データ化して利用する。
- 画像データとテキスト（メタデータ）や数値

データを同時に説明変数として利用する．（マルチモーダル学習）

このうち古くから行われていたのが，先日実習した1の方法である．今回は2について実習する．

3 画像を説明変数とする方法

1の方法では，画素ごとに数値化して，それを左上から右下の画素に向けて一列に並べるようにして説明変数とする．このため，2次元空間における上下左右の関係が壊れてしまう．

2の方法では，2次元空間における上下左右の関係を保持したまま，空間における画像の特徴量を扱うことができる．ただ，それがどのような特徴量であるかわかりにくい（説明しにくい）ことがある．

4 ライブラリの準備

CNNを利用する場合，ライブラリを利用する．
ここではpytorchを利用することとする．
環境を整備するためにインストールする必要がある．

1 `https://pytorch.org/`

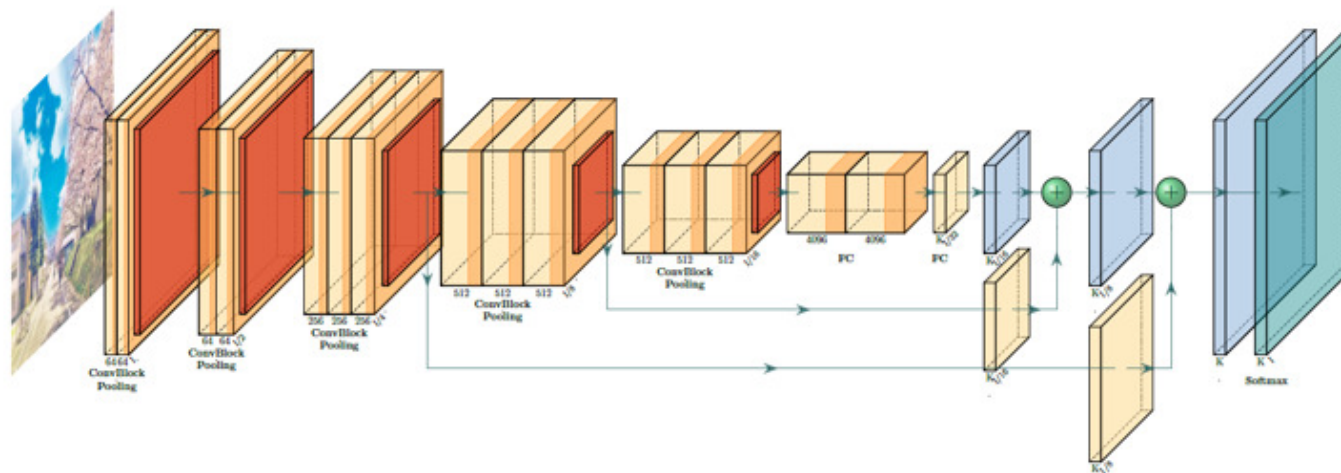
へアクセスするとインストール方法が示される．
例えば以下のようになる．

1 `pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126`

5 畳み込みニューラルネットワーク (CNN)

畳み込みニューラルネットワーク (CNN) は、受容フィールドと呼ばれる生物学的な受容メカニズムに着想を得ている。CNNは、一般に、畳み込み層、プーリング層、および全結合層を積み重ねることで定義されている。

6 畳み込みニューラルネットワーク (CNN) の例



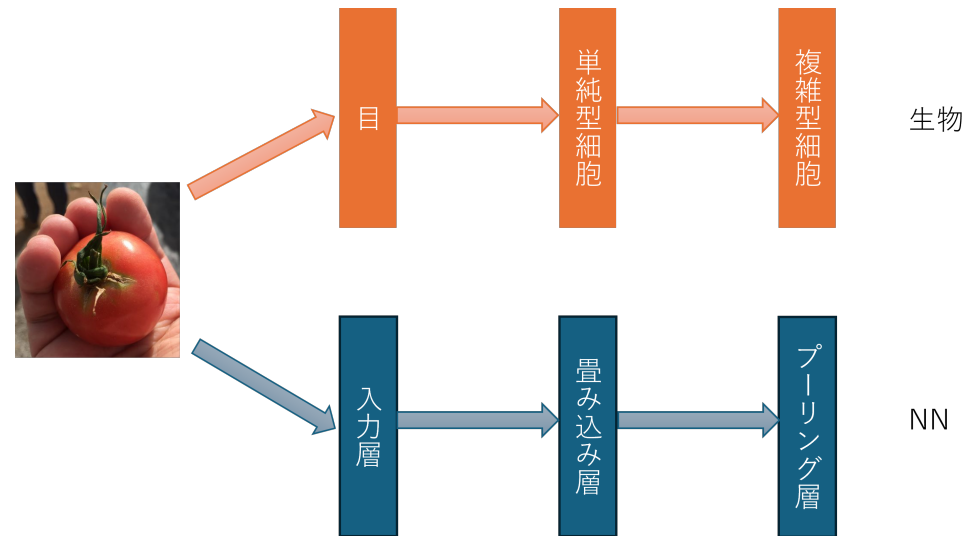
7 生物学的なメカニズムとCNN の関係

生物学的なメカニズムとCNNの関係を対比すると以下のようなになる．

生命の脳の視覚野の細胞において，部分の特徴を捉える機能と全体の特徴を捉える機能がある．

CNNにおいては，畳み込み層とプーリング層がそれを担当している．

8 生物学的なメカニズムとCNN の関係



9 畳み込み層 (Convolutional Layer)

- 画像処理においては，畳み込み処理によって画像の特徴量を抽出する．
- 畳み込み層は，画像の空間連続性を維持し，畳み込みフィルターを通して，画像の局所的な特徴を抽出する．
- 畳み込み操作後の画像は，特徴マップ (feature map) と呼ばれる．

10 畳み込みフィルタ

- エッジ検出フィルタ
- シャープニング フィルタ
- スムージング フィルタ
- 平滑化フィルタ
- 微分フィルタ（ソーベルフィルタ）

11 プーリング層 (Pooling Layer)

- プーリング層（サブサンプリング層）は，機能の数やパラメーターの数を減らし，過剰適合を制御する．
- 最大値プーリング
カーネル内の最大値だけを残すプーリング
- 平均プーリング
カーネル内の平均値を出力するプーリング

12 全結合層 (Fully Connected Layer)

- 全結合層の前層の出力と，後層の入力を相互接続する．
- 前層の情報を統合して抽象化を行う．

13 ドロップアウト (Dropout)

- ニューラルネットワークの学習において、一定数のノードをランダムに不活性化して学習する方法である。
- (学習データに対してだけ高い精度を示し、テストデータについて十分な精度を実現できない) 過学習と呼ばれる現象を防ぐことを目的としている。

14 バッチ正規化 (Batch Normalization)

- 正規化（ノーマライゼーション）とは，データの統計量（平均・分散）を平均0，分散1に変換することである．
- 学習が安定し，パラメータの影響が小さくなる．
- 学習データとテストデータの分布が異なる場合に重要となる．

15 CNNによってMNISTを判別 するプログラム例

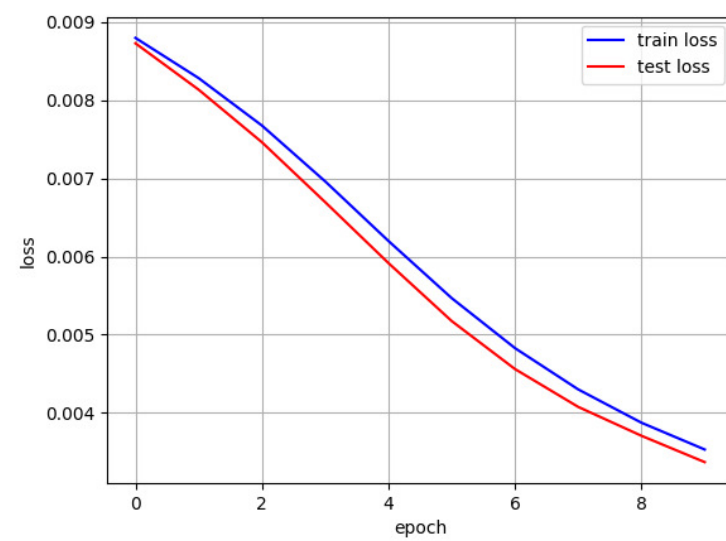
```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as f
4 import torchvision
5 import torchvision.transforms as transforms
6 import matplotlib.pyplot as plt
7
8 def load_MNISTtrain(batch):
9     transform = transforms.Compose(
10         [transforms.ToTensor(), transforms.Normalize((0.1307,),
11             (0.3081,))])
12     train_set = torchvision.datasets.MNIST(
13         root="./data", train=True, download=True, transform=
14         transform)
15     train_loader = torch.utils.data.DataLoader(
16         train_set, batch_size=batch, shuffle=True, num_workers=2)
17     return train_loader
```

```
16
17 def load_MNISTtest(batch):
18     transform = transforms.Compose(
19         [transforms.ToTensor(), transforms.Normalize
20          ((0.1307,), (0.3081,))]
21     )
22     test_set = torchvision.datasets.MNIST(
23         root="./data", train=False, download=True, transform=
24         transform)
25     test_loader = torch.utils.data.DataLoader(
26         test_set, batch_size=batch, shuffle=True, num_workers=2)
27     return test_loader
28
29 class NNet(nn.Module):
30     def __init__(self):
31         super(NNet, self).__init__()
32         self.conv1 = nn.Conv2d(1, 32, 3, 1)
33         self.conv2 = nn.Conv2d(32, 64, 3, 1)
34         self.pool = nn.MaxPool2d(2, 2)
35         self.fc = nn.Linear(12*12*64, 128)
36
37     def forward(self, x):
38         x = self.conv1(x)
```

```
36         x = f.relu(x)
37         x = self.conv2(x)
38         x = f.relu(x)
39         x = self.pool(x)
40         x = x.view(-1,12*12*64)
41         x = self.fc(x)
42         return f.log_softmax(x, dim=1)
43
44 def main():
45     epoch = 3
46     batch_size = 64
47
48     data_train = load_MNISTtrain(batch=batch_size)
49     data_test  = load_MNISTtest(batch=batch_size)
50
51     device = torch.device('cuda' if torch.cuda.is_available() else
52                           'cpu')
53     print(device)
54     model = NNet().to(device)
55     print(model)
56     criterion = nn.CrossEntropyLoss() # f.nll_loss()
57     print(criterion)
```

```
57 optimizer = torch.optim.Adam(params=model.parameters(), lr
    =0.001)
58 print(optimizer)
59
60 for e in range(epoch):
61
62     model.train()
63     loss = None
64     train_loss = 0.0
65     count = 0
66     for i,(data,label) in enumerate(data_train):
67         data,label = data.to(device),label.to(device)
68         optimizer.zero_grad()
69         output = model(data)
70         loss = criterion(output,label)
71         train_loss += loss.item()
72         count += 1
73         loss.backward()
74         optimizer.step()
75
76         if i % 100 == 99:
```

```
77         print("Training: {} epoch. {} iteration. Loss: {}".  
78               format(e+1,i+1,loss.item()))  
79     train_loss /= count  
80  
81     model.eval()  
82     test_loss = 0.0  
83     count = 0  
84     with torch.no_grad():  
85         for data,label in data_test:  
86             data,label = data.to(device),label.to(device)  
87             output = model(data)  
88             loss = criterion(output,label)  
89             test_loss += loss.item()  
90             count += 1  
91             predict = output.argmax(dim=1,keepdim=True)  
92  
93     test_loss /= count  
94  
95     print("Training loss (ave.): {}   Test loss: {} \n".format(  
96           train_loss,test_loss))  
  
97 if __name__ == "__main__":  
98     main()
```



16 プログラム例の説明(1)

- 8行目から24行目：MNISTデータをダウンロードする関数の定義
- 26行目から42行目：ネットワークの定義．畳み込み層が2層，プーリング層が1層となっている．活性化関数はReLU関数である．
- 45行目から58行目：パラメータ，モデル，損失関数と最適化法等を定義する．
- 62行目から78行目：モデル学習の定義
- 80行目から91行目：モデルテストの定義

17 プログラム例の説明(2)

プログラムにおいて，26行目から42行目の定義を変更すればネットワークを変更できる．また，45行目から58行目の損失関数と最適化法の定義も必要に応じて変更する．

18 演習問題

- Epoch数やバッチサイズを4倍に変更しなさい。
- ネットワークにドロップアウトを取り入れるため、構造を以下と変更して実施しなさい。

```
1 class NNet(nn.Module):
2     def __init__(self):
3         super(NNet,self).__init__()
4         self.conv1 = nn.Conv2d(1,32,3,1)
5         self.conv2 = nn.Conv2d(32,64,3,1)
6         self.pool = nn.MaxPool2d(2,2)
7         self.dropout = nn.Dropout2d(0.25)
8         self.fc = nn.Linear(12*12*64,128)
9
10    def forward(self,x):
11        x = self.conv1(x)
```

```
12         x = f.relu(x)
13         x = self.conv2(x)
14         x = f.relu(x)
15         x = self.pool(x)
16         x = self.dropout(x)
17         x = x.view(-1,12*12*64)
18         x = self.fc(x)
19         return f.log_softmax(x, dim=1)
```

- Dropoutの導入による計算精度と計算コストの変化を確認しなさい。
- （努力問題） 損失関数の収束曲線を記載しなさい。